



UNIVERSIDADE ESTADUAL PAULISTA
“JÚLIO DE MESQUITA FILHO”
Instituto de Ciência e Tecnologia
Câmpus de Sorocaba

PHELIPE ROCHA

SISTEMA DE MONITORAMENTO DE SENSORES INDUSTRIAIS COM
MÚLTIPLAS ESTRUTURAS DE DADOS:
IMPLEMENTAÇÃO E ANÁLISE EXPERIMENTAL DE DESEMPENHO EM C++

Sorocaba

2026

RESUMO

Este trabalho apresenta um código de um projeto para implementação e a análise de um simulador de monitoramento de sensores industriais de temperatura. Esse sistema é capaz de gerenciar um fluxo contínuo de leituras de temperatura que são geradas por sensores e esses registros são feitos em tempo real e simultâneo. Desse modo, permite a consulta desses registros por múltiplas estruturas de dados e as consultas podem ser interativa, quando necessário ocorre a remoção de elementos e mantém a indexação atualizado conforme a execução. O código foi feito em C++, que integra cinco estruturas de dados distintas que são: lista encadeada, árvore AVL e tabela hash. Contém duas estruturas avançadas que são: Bloom Filter e Fenwick Tree. Dessas estruturas, possui cada uma responsável por uma função específica do sistema. O objetivo é demonstrar as estruturas de dados aplicadas, evidenciando o comportamento teórico de cada uma sob diferentes cenários de uso. O sistema possui a geração sintética de um conjunto de dez mil leituras e a execução de operações de inserção, busca e remoção, a detecção estatística de anomalias pela regra do desvio padrão e a realização de testes em condições restritivas de memória, processamento, latência e qualidade de dados. O benchmark padronizado reúne os dados de tempo de execução por operação, taxa de colisões, uso de memória e escalabilidade. Os resultados logarítmico da árvore AVL com o crescimento do volume de dados e o ganho de localidade de referência pela otimização baseada em memory pool aplicada à lista encadeada. Posto isso essas estruturas, ao ser utilizada de forma combinada garante o processamento dos dados que foram coletados das medições realizadas em tempo real.

Palavras-chave: estruturas de dados; árvore AVL; tabela hash; Bloom Filter; análise de desempenho.

ABSTRACT

This work presents the implementation and analysis of an industrial temperature sensor monitoring simulator. The system is capable of managing a continuous stream of temperature readings generated by sensors and processing them in real time. It allows data queries through multiple data structures while maintaining updated indexes during execution. Developed in C++, the system integrates linked lists, an AVL tree, a hash table, a Bloom Filter, and a Fenwick Tree, each responsible for a specific function within the application. The objective is to demonstrate the practical application of data structures and evaluate their theoretical behavior under different usage scenarios. The system generates a synthetic dataset containing ten thousand readings and performs insertion, search, and deletion operations, statistical anomaly detection using the three-sigma rule, and tests under restrictive conditions involving memory, processing, latency, and data quality. Benchmarking collects execution time, collision rates, memory consumption, and scalability metrics. The results highlight the logarithmic behavior of the AVL tree and the improvement in memory locality achieved through the memory pool optimization applied to the linked list implementation.

Keywords: data structures; AVL tree; hash table; Bloom Filter; performance analysis.

SUMÁRIO

1 INTRODUÇÃO	5
2 FUNDAMENTAÇÃO TEÓRICA	5
2.1 Estruturas de dados	5
3 METODOLOGIA E ARQUITETURA DO SISTEMA.....	6
3.1 Organização dos arquivos.....	7
3.2 Escolha da linguagem	7
3.3 Estrutura central de dados	7
4 IMPLEMENTAÇÃO DAS ESTRUTURAS DE DADOS	8
4.1 Árvore AVL.....	8
4.2 Tabela hash	9
4.3 Lista encadeada.....	9
4.4 Bloom Filter	10
4.5 Fenwick Tree.....	10
5 OPERAÇÕES ADICIONAIS E OTIMIZAÇÃO DE MEMÓRIA	10
5.1 Geração e tratamento do conjunto de dados	10
5.2 Operações adicionais de processamento	11
5.3 Otimização baseada em memory pool	11
5.4 Testes em condições restritivas	11
6 ANÁLISE EXPERIMENTAL E DE COMPLEXIDADE	12
6.1 Critério de comparabilidade	12
6.2 Análise de complexidade computacional	12
6.3 Resultados observados.....	13
7 CONCLUSÃO	13
REFERÊNCIAS.....	14

1 INTRODUÇÃO

O desenvolvimento de sistemas capazes de manipular grandes volumes de dados em tempo real (10000 dados) faz com que seja necessário uso de estruturas de dados mais adequadas a cada tipo de operação.

Esse monitoramento contínuo de sensores é um código usado para obter em tempo real essas medidas e manipulação desses dados obtidos. Foi proposto por esse trabalho a construção de um código capaz de realizar o monitoramento de sensores industriais de temperatura, capaz de receber um fluxo contínuo de leituras, organizar em múltiplas estruturas, responder a consultas interativas e manter seus índices internos atualizados durante toda a execução.

Com isso, o sistema implementa cinco estruturas distintas e realiza por meio de medição de tempo o comportamento e a velocidade de busca e remoção de dados. Foram implementadas estruturas: lista encadeada dinâmica, lista encadeada estática com Memory Pool, Tabela Hash, Árvore AVL, Fenwick Tree e Bloom Filter. Cada estrutura foi associada a uma função específica do sistema. A Tabela Hash mantém o estado atual dos sensores; a Árvore AVL indexa o histórico cronológico; a Fenwick Tree realiza consultas de intervalo; o Bloom Filter executa triagem probabilística de sensores críticos e as listas armazenam o histórico recente e permitem a comparação entre uma versão tradicional e uma versão otimizada.

Foi proposto a implementação manualmente cada estrutura, sem recorrer às coleções prontas da biblioteca-padrão e integrar todas elas a um mesmo domínio de aplicação, também aplicar técnicas de tratamento de erros e aumento de dados na geração do conjunto de leituras. Somado a isso, submissão do sistema a testes em condições restritivas.

2.1 Estruturas de dados

Uma **lista encadeada** é uma lista que consiste em vários blocos de memória separados. Cada bloco de memória contém um valor e um ponteiro para o próximo bloco de memória contendo o próximo valor (ou um ponteiro nulo ou equivalente marcando o fim da lista).

Isso é diferente de uma lista de array, que armazena todos os valores juntos em um único bloco de memória. A vantagem da sua utilização é a inserção e a

remoção eficientes em posições conhecidas; em contrapartida, o acesso a um elemento arbitrário exige que seja percorrido a sequência desde o início.

A **árvore AVL** é um tipo de árvore binária de busca tem o autobalanceamento dinâmico. Uma árvore binária de busca é compostas de nós. Ela tem as seguintes características: cada árvore possui um nó raiz, esse nó raiz tem filhos e esses filhos seguem a estrutura de ter até dois filhos. Para cada nó, seus descendentes da esquerda são menores que o nó atual, que é menor que o descendentes da direita. Para cada nó, ela mantém a diferença de altura entre as subárvores esquerda e direita — o fator de balanceamento — restrita ao intervalo de menos um a mais um. A implementação de uma AVL incluirá um algoritmo para balanceamento da árvore quando a adição de um elemento prejudicar essa garantia.

As árvores AVL têm uma complexidade de tempo de busca, inserção e exclusão, no pior dos casos, de $O(\log n)$, onde n é o número de nós na árvore. A complexidade de espaço de pior caso é $O(n)$.

Esse equilíbrio é preservado por meio de rotações executadas após inserções e remoções, garantindo altura logarítmica e, por consequência, operações de busca, inserção e remoção em $O(\log N)$.

A **tabela hash** é uma função que a partir do momento que insere dados ou e feito uma leitura desses dados, gera um valor numérico que esta atrelado a posição de um elemento nessa tabela. A função faz o mapeamento das posições de um arranjo por meio de uma função de dispersão tendo um acesso a esse dado, essa busca em $O(1)$ é em tempo constante independente do tamanho da tabela. A escolha dessa tabela é justamente por conseguir percorrer de forma constante independente do tamanho da tabela. Ela percorre o array e cria um índice, com isso ela localiza a palavra e retorna um número que representa a palavra deseja. O contratempo são as colisões que ocorre quando duas palavras chaves de pesquisa que são diferentes tem o mesmo valor hash, esse valor é a posição. Porque na função hash ela gera um valor que representa a chave de pesquisa e essas palavras chaves são muito maiores do que os números de valores gerados hash. Para evitar essas colisões, foi recorrido ao auxilio da IA um endereçamento aberto com sondagem linear, existem também o encadeamento, porém se duas chaves de pesquisa diferentes têm o mesmo valor hash. Com isso, se torna ineficiente para espaço de memória ocupado e desempenho que é um dos objetivos desse projeto. O endereçamento aberto por sua vez, se duas

palavras chaves tem o mesmo valor hash, a segunda é submetida a uma posição na tabela que ainda não foi ocupada, mas é complexo de ser implementado.

Bloom Filter é uma estrutura probabilística para teste de pertinência. O seu objetivo é demonstrar se um elemento existe dentro da tabela de dados. Essa tabela não precisa ser totalmente armazenada, desse modo utiliza uma quantidade bem menor de espaço em memória. É formado por um vetor de bits e múltiplas funções hash independentes e ao realizar as consultas guarda o resultado obtido por cada uma das funções Hash. Sua característica fundamental é nunca produzir falsos negativos: se a estrutura responde que um elemento não pertence ao conjunto, essa resposta é sempre correta, nesse contexto nunca irá trazer um elemento que não existe na tabela de dados sendo que na verdade esse elemento existe. Porém possui falsos positivos, com grande economia de memória, mas não é possível excluir um elemento do bloomfilter uma vez que ele foi incluído, se precisar fazer sua remoção, precisa ser gerado novamente.

A **Fenwick Tree**, também conhecida como árvore de Fenwick, ou Binary Indexed Tree permite calcular somas de prefixo e atualizar valores, ambas as operações em $O(\log N)$. A sua utilização é de maneira simples de ser implementada principalmente nos casos de matrizes multidimensional de dados. Tem como base a manipulação dos bits dos índices, dispensando o armazenamento explícito de ponteiros entre nós.

3 METODOLOGIA E ARQUITETURA DO SISTEMA

O sistema foi desenvolvido em C++, tendo implementação de cada estrutura de dados em um arquivo de cabeçalho independente e no main a concentração de todas essas funções de dados.

3.1 Organização dos arquivos

O Quadro 1 resume a distribuição de responsabilidades entre os arquivos do projeto.

Quadro 1 — Organização dos arquivos do projeto

Arquivo	Conteúdo
main.cpp	Organização do conjunto de dados, menu, benchmarks e testes restritivos

Arquivo	Conteúdo
ArvoreAVL.hpp	Implementação da árvore AVL
TabelaHash.hpp	Implementação da tabela hash
bloomfilter.hpp	Implementação do Bloom Filter
FenwickTree.hpp	Implementação da Fenwick Tree

Fonte: elaborado pelo autor.

3.2 Escolha da linguagem

A linguagem C++ foi escolhida por sugestão direta do professor no manual a seguir e também por oferecer controle direto sobre a alocação de memória e o acesso a ponteiros que é extremamente necessário tanto para a implementação manual das estruturas sem o uso das coleções da biblioteca-padrão, quanto para a otimização baseada em memory pool que influencia no processador.

3.3 Estrutura central de dados

A unidade fundamental de informação é a estrutura `LeituraSensor`, composta por três campos: o identificador do sensor (`id_sensor`), uma marca temporal (`timestamp`) e o valor medido (`valor`). Essa estrutura trafega entre todas as cinco classes.

4 IMPLEMENTAÇÃO DAS ESTRUTURAS DE DADOS

4.1 Árvore AVL

A árvore AVL é um tipo de árvore binária de busca com o autobalanceamento dinâmico. Uma árvore binária de busca é composta de nós. Para gerenciar, utilizou-se o `timestamp` como chave de ordenação porque ele é único para cada leitura e gerado sequencialmente, conforme os identificadores de sensor repetem a medição. Como a árvore não admite chaves duplicadas, o `timestamp` garante que somente seja gerado um e viabiliza buscas cronológicas.

A inserção mantém a ordenação e, ao final, reequilibra a árvore por meio de rotações. A busca percorre a árvore comparando para evitar erros de igualdade exata entre números em ponto flutuante. Essa remoção trata três casos: nó folha, nó com um único filho e nó com dois filhos. O balanceamento é assegurado pelas rotações

simples e duplas, que cobrem os quatro casos de desequilíbrio conhecidos pelas siglas LL, RR, LR e RL.

4.2 Tabela hash

A TabelaHash implementa uma tabela de dispersão com endereçamento aberto. O arranjo interno possui tamanho fixo de 1009 posições, um número primo escolhido para distribuir melhor as chaves e reduzir agrupamentos. Foi escolhido para reduzir as colisões porque se a entrada tiver um padrão ou regularidade, as colisões fará presentes.

O tratamento de colisões utiliza sondagem linear, dessa maneira quando a posição calculada já está ocupada, a estrutura avança sequencialmente até encontrar um espaço livre, mantendo um contador de colisões, conforme solicitado no documento estruturas disponibilizado pelo professor afim de fazer comparação. Esse cuidado preserva a cadeia de sondagem, pois um slot simplesmente esvaziado poderia interromper buscas posteriores que deveriam prosseguir além daquela posição. Slots marcados como removidos podem ser reaproveitados em inserções futuras, ou seja não é excluído e perdido esses dados.

4.3 Lista encadeada

Existem duas listas para fins de comparação, primeiro a versão dinâmica aloca cada nó individualmente no heap, por meio do operador new, resultando em endereços dispersos pela memória. A versão estática, otimizada, mantém todos os nós em um único arranjo contíguo pré-alocado e substitui os ponteiros por índices inteiros. Ambas implementam inserção, busca e remoção; o contraste de desempenho entre elas é analisado na Seção 5.

4.4 Bloom Filter

A classe BloomFilter implementa o filtro probabilístico utilizando um arranjo de bytes manipulado bit a bit, é mais previsível do que uma biblioteca de uso padrão. São empregadas três funções hash independentes, tendo objetivo de reduzir falsos positivos. A estrutura não oferece remoção como um mesmo bit pode pertencer a vários elementos ao removê-lo poderia produzir um falso negativo.

4.5 Fenwick Tree

A classe FenwickTree é um tipo de árvore calcula somas de prefixo de forma eficiente. Tanto a atualização quanto a consulta de prefixo operam em $O(\log N)$, por meio da operação $i \& (-i)$, que isola o bit menos significativo do índice e determina os saltos a serem realizados na estrutura. A consulta de intervalo obtém a soma de uma faixa pela subtração de duas somas de prefixo. A estrutura adota indexação iniciada em um, pois a manipulação de bits não opera corretamente sobre o índice zero.

5 OPERAÇÕES ADICIONAIS E OTIMIZAÇÃO DE MEMÓRIA

5.1 Geração e tratamento do conjunto de dados

Os dados são gerados no arquivo principal chamado main.cpp, já o dataset utilizado é sintético e contém (10)dez mil leituras de sensores de temperatura. As leituras são produzidas por uma distribuição normal, com média de 24 °C e desvio-padrão de 2 °C, simulando as flutuações naturais de um sensor real; essa técnica de aumento de dados confere realismo ao experimento. Para validar o mecanismo de detecção de anomalias, valores atípicos são injetados em pontos específicos do fluxo, associados a identificadores de sensor distintos. O volume de dez mil amostras é suficiente para evidenciar diferenças mensuráveis de desempenho: a árvore AVL atinge altura próxima de quatorze ao realizar a conta com log, compatível com o logaritmo de base dois de dez mil, e a tabela hash de tamanho fixo ultrapassa seu fator de carga seguro, expondo o comportamento de saturação para verificar o comportamento no sistema.

```
Indexando fluxo contínuo industrial (10.000 amostras)...
Sistema carregado! Média=23.97 °C | Desvio=2.04 °C
```

PAINEL OPERACIONAL DE SENSORES INDUSTRIAIS	
[1] Estado atual dos sensores	(Tabela Hash · $O(1)$)
[2] Busca histórica por timestamp	(AVL · $O(\log N)$)
[3] Média térmica por intervalo	(Fenwick Tree · $O(\log N)$)
[4] Triagem de segurança	(Bloom Filter · $O(1)$)
[5] Histórico recente	(Lista Pool Estática)
[6] Remover sensor do sistema	(CRUD · todas estruturas)
[7] Benchmarks C7/C9 (Listas · Hash · AVL · Escalabilidade)	
[8] Cenários restritivos C8 (R4/R7/R12/R18/R19/R21)	
[0] Encerrar sistema	

Fonte: Imagem de autoria própria do código

5.2 Operações adicionais de processamento

Além das operações básicas, o sistema executa três operações adicionais. A **detecção de anomalias**, implementada pela classe `DetectorAnomalias`, aplica a regra dos três desvios-padrão: durante o treinamento, calcula a média e o desvio-padrão das leituras e, em seguida, classifica como anômala qualquer leitura que se afaste da média por mais de três desvios-padrão limiar que, estatisticamente, abrange cerca de 99,7% dos dados normais. A **consulta de intervalo** utiliza a Fenwick Tree para calcular, em tempo logarítmico, a soma e, por extensão, a média das leituras em uma janela temporal arbitrária. A **triagem probabilística**, baseada no Bloom Filter, identifica instantaneamente sensores que nunca dispararam alertas críticos, evitando buscas sem necessidade.

5.3 Otimização baseada em memory pool

A estrutura escolhida para otimização foi a lista encadeada. A ideia foi transformar uma versão tradicional, com alocação dinâmica espalhada pela memória, em uma versão mais compacta, utilizando um memory pool. Na implementação convencional, cada nó da lista é alocado separadamente, o que faz com que fiquem em posições aleatórias e distantes na memória. Já na versão otimizada, todos os nós ficam armazenados em um único vetor pré-alocado lado a lado, e os ponteiros são substituídos por índices inteiros que indicam a posição de cada nó nesse vetor.

O principal benefício dessa mudança está na localidade de referência. Em uma lista dinâmica comum, quando o programa passa de um nó para outro, o processador precisa acessar diferentes regiões da memória, o que aumenta a chance de falhas de cache e reduz o desempenho. Com o uso do memory pool, como os nós estão próximos uns dos outros, o processador consegue aproveitar melhor o cache e o processador acessa os próximos dados com mais facilidade, tornando a leitura da lista mais rápida. Além disso, há também uma economia de memória. Em arquiteturas de 64 bits, um ponteiro geralmente ocupa oito bytes, enquanto um

inteiro pode ocupar apenas quatro bytes. Da mesma forma, a memória é reservada uma única vez, evitando o trabalho extra de criar cada nó separadamente.

5.4 Testes em condições restritivas

O sistema implementa seis cenários restritivos, desde as categorias de memória, processamento, latência, dados e algorítmica, trazendo registrado no Quadro 2.

Quadro 2 — Cenários de teste em condições restritivas

Cenário	Categoria	Descrição
R4	Memória	Comparação de localidade de cache entre o pool e o heap
R7	Processamento	Orçamento fixo de 5 μ s por operação, com contagem de conformidade
R12	Latência	Injeção de atrasos aleatórios (0–2 μ s), simulando jitter de rede
R18	Dados	Injeção de 10% de leituras anômalas, com medição da taxa de detecção
R19	Dados	Truncamento de resolução (double para int), com cálculo do erro médio
R21	Algorítmica	Substituição da busca AVL $O(\log N)$ por busca linear $O(N)$

Fonte: elaborado pelo autor.

Todos os cenários resultam em medições reais durante a execução. Sendo assim, possibilita observação direta do impacto de cada restrição e mecanismos de busca sobre o comportamento das estruturas.

6 ANÁLISE EXPERIMENTAL E DE COMPLEXIDADE

Um módulo de benchmark, que é um padrão de comparação, padronizado por indicadores uniformes para todas as estruturas comparáveis: tempo de inserção, busca e remoção, medido em nanossegundos; taxa de colisões, específica da tabela hash, uso de memória, latência média de operações combinadas com a variação do volume de dados.

6.1 Critério de comparabilidade

A análise é feita com estruturas que contém a mesma finalidade e sendo assim podendo ser comparadas. Assim, os tempos de busca da tabela hash e da árvore AVL são comparados entre si, pois ambas resolvem o problema de busca por chave. Esse tipo de busca está descrito ao longo do projeto. Já o Bloom Filter, voltado ao teste de pertinência e a Fenwick Tree, voltada à soma de intervalos, têm finalidades distintas. Urge portanto, avaliados apenas por suas medições próprias. As medições não aplicáveis são explicitamente justificadas: a remoção, por exemplo, não se aplica ao Bloom Filter, pela limitação descrita ao longo do relatório.

6.2 Análise de complexidade computacional

As diferenças de complexidade explicam-se pelo modo de organização de cada estrutura. A busca na lista é $O(N)$ porque exige percorrer sequencialmente os nós até localizar o alvo. A árvore AVL alcança $O(\log N)$ porque o balanceamento mantém a altura proporcional ao logaritmo do número de elementos. A tabela hash atinge tempo médio constante quando o fator de carga é baixo, porém a $O(N)$ à medida que as colisões se acumulam. O Bloom Filter e a Fenwick Tree operam, respectivamente, em tempo constante no número de funções hash e em tempo logarítmico, conforme a manipulação de bits que as define.

6.3 Resultados observados

Os experimentos confirmaram a teoria prevista quanto a usabilidade de cada uma das estruturas. Por exemplo, a árvore AVL manteve altura próxima do logaritmo de base dois do número de elementos à medida que o volume crescia de cem a dez mil registros, sustentando o crescimento logarítmico esperado, mantendo o equilíbrio que é o que se espera da função árvore. Já a tabela hash apresentou tempo por operação praticamente constante enquanto o fator de carga é baixo, pois sua teoria diz que a razão pela qual sua medição foi limitada a setecentos elementos, abaixo do limiar de saturação. Na comparação entre as listas, a versão com memory pool apresentou travessia mais rápida do que a versão dinâmica, com isso o ganho de localidade de referência, além de menor consumo de memória pela substituição de ponteiros por índices inteiros. No cenário R21, a busca em árvore AVL mostrou-se substancialmente mais rápida do que a busca linear no pior caso, em conformidade com a diferença teórica entre $O(\log N)$ e $O(N)$, a equação descrita ao ser implementada comprova esse resultado.

7 CONCLUSÃO

Este trabalho apresentou a implementação e a comparação de cinco estruturas de dados aplicadas a um sistema de monitoramento de sensores industriais. Cada estrutura foi integrada a forma de aplicação: a tabela hash para o estado em tempo real, a árvore AVL para o índice cronológico, a Fenwick Tree para agregações por intervalo, o Bloom Filter para a triagem de segurança e a lista encadeada para o histórico recente e para a demonstração de otimização de memória.

Os experimentos comprovaram as características teóricas das estruturas, com destaque para o ganho de localidade de referência através da otimização utilizando memory pool e para a manutenção do comportamento logarítmico da árvore AVL conforme o crescimento do volume de dados. Isso é fundamental para a inserção e remoção de valores e comparação com tempo de busca.

REFERÊNCIAS

BLOOM, Burton H. Space/time trade-offs in hash coding with allowable errors. Communications of the ACM, New York, v. 13, n. 7, p. 422-426, 1970.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Algoritmos: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

FENWICK, Peter M. A new data structure for cumulative frequency tables. Software: Practice and Experience, Chichester, v. 24, n. 3, p. 327-336, 1994.

KNUTH, Donald E. The art of computer programming: sorting and searching. 2. ed. Reading: Addison-Wesley, 1998. v. 3.

ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2011.